Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Мисюрко В. А.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python.

Задание 1
Реализовать простой класс
Требования к выполнению
• Реализовать пользовательский класс по варианту. Для каждого класса
• Создать атрибуты (поля) классов
• Создать методы классов
• Добавить необходимые свойства и сеттеры (по необходимости)
• Переопределить магические методы __str__ и __eq__

4) Прямоугольник, заданный длинами двух сторон – Предусмотреть возможность определения площади и периметра, а также логические методы, определяющие, является ли прямоугольник квадратом и существует ли такой прямоугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

```
class Rectangle:
    def __init__(self, a: float, b: float):
        self.a = a
        self.b = b

    @property
    def a(self):
        return self._a

    @a.setter
    def a(self, value):
        if value <= 0:
            raise ValueError("Сторона должна быть положительным числом")
        self._a = value

    @property
    def b(self):
        return self._b

    @b.setter
    def b(self, value):
        if value <= 0:
            raise ValueError("Сторона должна быть положительным числом")
        self._b = value

    def area(self) -> float:
        return self.a * self.b

    def perimeter(self) -> float:
        return 2 * (self.a + self.b)

    def is_square(self) -> bool:
        return self.a == self.b

    def exists(self) -> bool:
        return self.a > 0 and self.b > 0

    # --- Магические методы ---

    def __str__(self):
        return f"Прямоугольник со сторонами {self.a} и {self.b}"

    def __eq__(self, other):
        if not isinstance(other, Rectangle):
```

```
            return False
        return (self.a == other.a and self.b == other.b) or \
               (self.a == other.b and self.b == other.a)

if __name__ == "__main__":
    r1 = Rectangle(5, 10)
    r2 = Rectangle(10, 5)

    print(r1)
    print("Площадь:", r1.area())
    print("Периметр:", r1.perimeter())
    print("Является квадратом:", r1.is_square())
    print("Существует:", r1.exists())
    print("Равны ли r1 и r2:", r1 == r2)
```

```
Прямоугольник со сторонами 5 и 10
Площадь: 50
Периметр: 30
Является квадратом: False
Существует: True
Равны ли r1 и r2: True
```

Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы

3) Система Больница. Пациенту назначается лечащий Врач. Врач может сделать назначение Пациенту (процедуры, лекарства, операции). Медсестра или другой Врач выполняют назначение. Пациент может быть выписан из Больницы по окончании лечения, при нарушении режима или иных обстоятельствах.

```
from abc import ABC, abstractmethod

class Person:
    def __init__(self, name: str):
        self.name = name

    def __str__(self):
        return f"{self.__class__.__name__}: {self.name}"

class MedicalStaff(Person):
    pass

class Doctor(MedicalStaff):
    def prescribe(self, patient, description: str):
        prescription = Prescription(description, self, patient)
        patient.add_prescription(prescription)
        return prescription

class Nurse(MedicalStaff):
    pass

class Patient(Person):
    def __init__(self, name: str):
        super().__init__(name)
        self.prescriptions = []
        self.is_discharged = False
        self.discharge_reason = None
```

```
    def add_prescription(self, prescription):
        self.prescriptions.append(prescription)

    def discharge(self, reason: str):
        self.is_discharged = True
        self.discharge_reason = reason

    def __str__(self):
        status = "Выписан" if self.is_discharged else "На лечении"
        return f"Пациент: {self.name}, статус: {status}"

class Executable(ABC):
    @abstractmethod
    def execute(self, performer):
        pass

class Prescription(Executable):
    def __init__(self, description: str, doctor: Doctor, patient: Patient):
        self.description = description
        self.doctor = doctor
        self.patient = patient
        self.is_executed = False

    def execute(self, performer):
        if not isinstance(performer, MedicalStaff):
            raise ValueError("Исполнитель должен быть медицинским
персоналом")
        self.is_executed = True
        print(f"{performer.name} выполнил назначение: {self.description}")

    def __str__(self):
        status = "Выполнено" if self.is_executed else "Не выполнено"
        return f"Назначение: {self.description} ({status})"

class Hospital:
    def __init__(self, name: str):
        self.name = name
        self.patients = []
        self.staff = []

    def add_patient(self, patient: Patient):
        self.patients.append(patient)

    def add_staff(self, staff_member: MedicalStaff):
        self.staff.append(staff_member)

    def __str__(self):
        return f"Больница: {self.name}, пациентов: {len(self.patients)},
персонала: {len(self.staff)}"

if __name__ == "__main__":
    hospital = Hospital("Городская больница")

    doctor = Doctor("Иванов")
    nurse = Nurse("Петрова")
    patient = Patient("Сидоров")

    hospital.add_staff(doctor)
    hospital.add_staff(nurse)
    hospital.add_patient(patient)

    print(hospital)
```

```python
# Врач назначает лечение
prescription = doctor.prescribe(patient, "Приём антибиотиков")

print(prescription)

# Медсестра выполняет назначение
prescription.execute(nurse)

print(prescription)

# Выписка пациента
patient.discharge("Окончание лечения")

print(patient)
```

```
Больница: Городская больница, пациентов: 1, персонала: 2
Назначение: Приём антибиотиков (Не выполнено)
Петрова выполнил назначение: Приём антибиотиков
Назначение: Приём антибиотиков (Выполнено)
Пациент: Сидоров, статус: Выписан
```

Вывод: закрепил навыки объектно-ориентированного программирования на языке Python.